

## 14 Interrupt Generation

A dedicated interrupt generation block is implemented which is handling requests from independent sources to generate an interrupt. The different requesters are as follows:

- State machine in case:
  - A requested state transition has not been performed successfully, e.g. SLEEP state could not be entered because of LDO\_μC current consumption above the selected threshold level.
  - A requested state transition from SLEEP has been performed successfully, i.e. the system has either successfully entered WAKE state. The uC may only send (additional) SPI commands after an interrupt event has been generated by the system. The purpose of the interrupt event is to inform the uC that a state transition has been performed successfully and that the system is capable of performing SPI communication at full SPI speed.
- Watchdog, an interrupt request is generated if the Watchdog is not serviced properly and configured in a way to allow service errors to occur, i.e. an error counter threshold value of more than 2 is configured. In this case an interrupt is generated only if the error counter threshold is not exceeded due to this error.
- Error pin monitoring, an interrupt request is generated if the error pin monitoring block detects an error and is configured in a way to allow occurrence of this error for a certain amount of time (recovery delay action enabled). In this case an interrupt is requested if an error is detected by the error pin monitoring and the recovery delay has not expired.
- Monitoring Block, an interrupt request is generated based on the defined system reaction described in [Chapter 11.4](#).
- Overtemperature warnings and over temperature shutdown of communication LDO
- Overcurrent conditions of voltage reference or standby LDO
- SPI block in case an SPI error has occurred
- ABIST operation has been completed
- Double bit error in the protected configuration

An interrupt is generated to inform a connected uC that a non-severe system condition has occurred. This allows the uC to perform proper action based on the source of the interrupt. A single interrupt line exists, which is high on default. All Internal interrupt sources are enabled by default and cannot be disabled.

An interrupt is signaled by pulling the interrupt line low for at least  $t_{INT}$  (interrupt min. pulse width) after an internal interrupt condition occurs. The interrupt line will be driven high if all of the **IF** register flag(s) has/have been cleared via SPI operation earliest after  $t_{INT}$  has expired but latest after  $t_{INTTO}$  has expired.

Special cases:

- If an interrupt is signaled by pulling INT low and not all interrupt status flags are cleared by the uC within  $t_{INTTO}$ , the INT will stay low until  $t_{INTTO}$  has expired, but no additional interrupt will be generated. Information about a pending interrupt event can be derived via the INTMISS status flag. This status flag is cleared each time the interrupt line is driven low.
- If an interrupt is signaled by pulling INT low and an additional bit is set in the **IF** register interrupt flag after the interrupt bits have been read by the uC and this outdated information is used to clear the interrupt flags, the interrupt line will stay low until  $t_{INTTO}$  has expired, but no additional interrupt will be generated. Information about a pending interrupt event can be derived via a status flag.
- After releasing the interrupt line to high, the interrupt line will stay high for at least  $t_{INTTO}$  regardless if any additional internal interrupt condition has occurred or not. If a new interrupt event occurs during the delay time out ( $t_{INTTO}$ ), this will be signaled by generating a new pulse after the delay time out  $t_{INTTO}$ .

All interrupt sources can only be cleared by a “write-1-to-clear” (w1c) SPI operation, i.e. writing a logic one to the corresponding bit(s) in the interrupt register will clear the event.

Interrupt events are organized in a two level approach. The first level (interrupt flag) provides information about different groups of interrupt events. The second level (status flags) provides detailed information about which particular event(s) generated the interrupt. To service an interrupt one would only need to write the interrupt flag register (**IF**). The status flag registers are only meant to provide detailed information. However all status flags can be cleared as well.

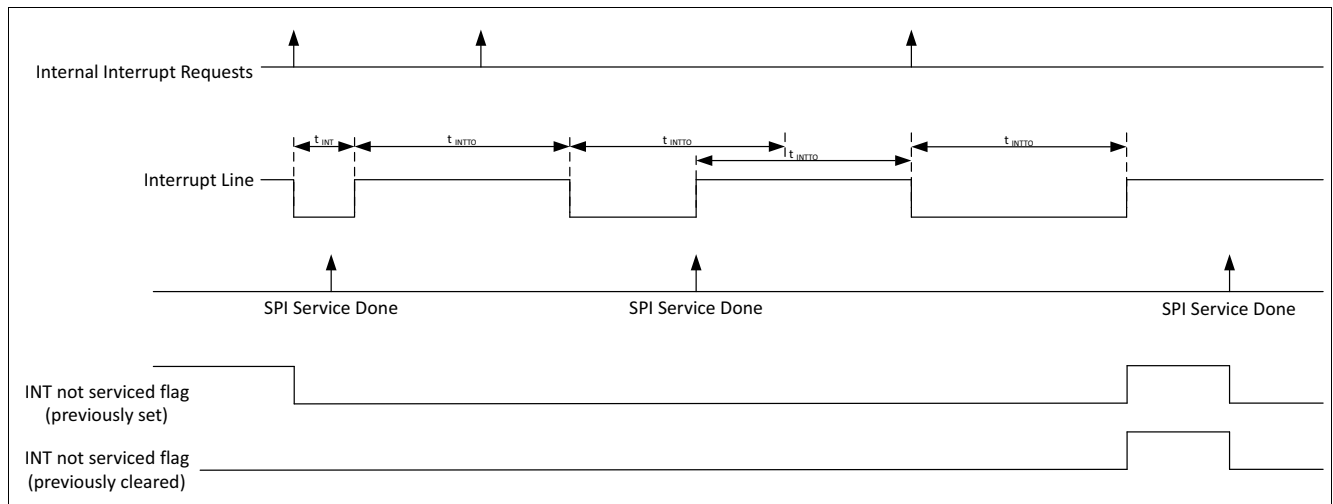
## Recommended interrupt service routine:

After an interrupt has been detected by an uC, the recommended interrupt service routine would need to perform the following tasks via SPI:

1. Read interrupt flag register (**IF**)
2. Read status flag register(s) based on information from first read
3. Take the proper action based on interrupt flag(s) and status flag(s)
4. Write back the status flag register(s) to clear particular status flag(s)
5. Write back the interrupt flag register (**IF**) with the previously read value to clear
6. Recommended: Read interrupt flag register (**IF**) again to check for occurrence of another interrupt event. In case step back to 2).
7. Writing back the interrupt register (**IF**) will release the interrupt line INT if all bits are cleared (interrupt timing requirements will be fulfilled).

An interrupt is only generated after the reset signal to the uC has been released. An interrupt event which occurred while the reset line for the uC is still active is not signaled at the interrupt line but the particular status bit for this event is set.

Details about the timing of the interrupt line are depicted in the following figure:



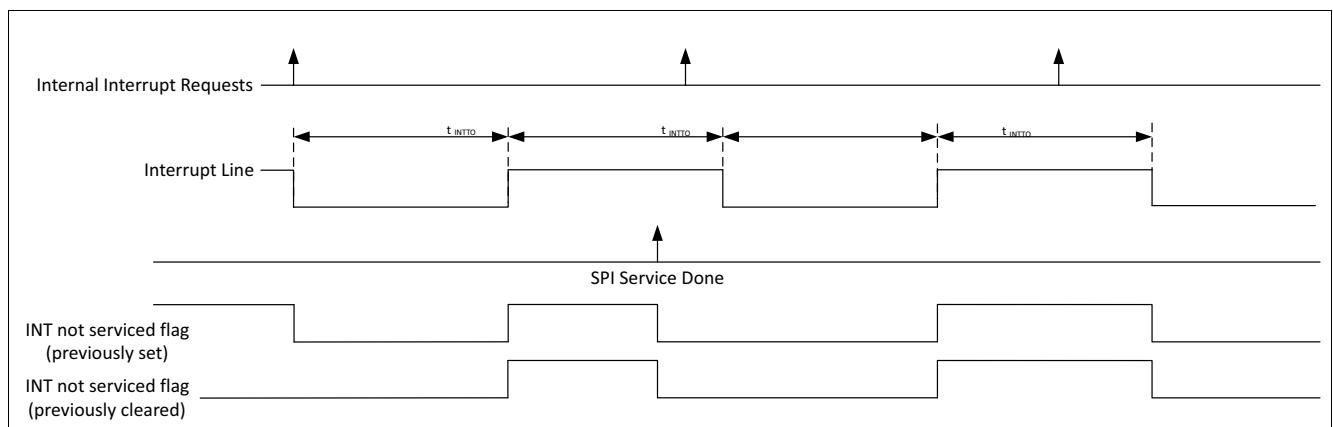
**Figure 76** Interrupt timing

**Table 24 Electrical Characteristics: Interrupt - Timings**

$V_{VS} = 6.0 \text{ V to } 40 \text{ V}$ ,  $T_j = -40^\circ\text{C to } +150^\circ\text{C}$ , all voltages with respect to ground, positive current flowing into pin (unless otherwise specified)

| Parameter                     | Symbol        | Values |      |      | Unit          | Note / Test Condition | Number |
|-------------------------------|---------------|--------|------|------|---------------|-----------------------|--------|
|                               |               | Min.   | Typ. | Max. |               |                       |        |
| Minimum interrupt pulse width | $t_{INT,min}$ | 90     | 100  | 150  | $\mu\text{s}$ |                       | P_14.1 |
| Interrupt time out            | $t_{INTTO}$   | 270    | 300  | 350  | $\mu\text{s}$ |                       | P_14.2 |

Details about the system behavior in case not all interrupt status flags have been cleared are depicted in the following figure:

**Figure 77 Interrupt timing without service in time**

An interrupt is always generated if an internal status flag is set, irrespective of the current state of this status flag. For example, if a parity error on SPI communication is detected for the first time, the flag SPISF.PARE is set and an interrupt is generated. This flag will also set the corresponding SPI interrupt flag (IF.SPI). If the flag IF.SPI is not cleared, but a second SPI parity error occurs, a new interrupt will be generated regardless. The timings shown in the two figures above will be fulfilled for generating a new interrupt.